Ferramenta Baseada em Linguagem de Domínio Específico para o Apoio ao Ensino de Autômatos Finitos com Simulação Visual Interativa

João Thomaz Vieira Klaus Siegfried Beckmann

Bacharelado em Ciência da Computação
Universidade Tuiuti do Paraná
Orientador: Prof. Diógenes Cogo Furlan

Curitiba, 2026

Roteiro

1. Introdução
2. Fundamentação Teórica
3. Trabalhos Relacionados
4. Metodologia de Construção da Solução
5. Ferramentas para Implementação
6. Projeto da DSL
7. Implementação
8. Resultados e Discussões
9. Conclusão

Roteiro

1. Introdução
2. Fundamentação Teórica
3. Trabalhos Relacionados
4. Metodologia de Construção da Solução
5. Ferramentas para Implementação
6. Projeto da DSL
7. Implementação
8. Resultados e Discussões
9. Conclusão

Contexto e problema

- Autômatos finitos são um pilar da Ciência da Computação, mas a disciplina é **reconhecidamente difícil**.
- Morazán e Antunez (2014): o estudante projeta autômatos **sem poder experimentar** nem receber **retroalimentação imediata**, ao contrário da programação.
- As ferramentas mais difundidas (**JFLAP**) são **só visuais**: sem descrição textual, sem versionamento, **sem diagnóstico** de erros.
- A alternativa textual (**FSM**, em Racket) descreve autômatos como código, mas **não os desenha nem anima**.

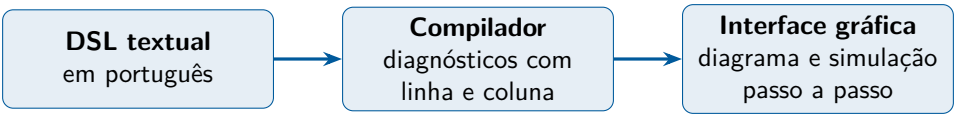
Lacuna

Unir a **descrição textual**, o **diagnóstico de um compilador** e a **simulação visual** em uma única ferramenta.

Pergunta de pesquisa e proposta

Pergunta de pesquisa

Como uma linguagem textual de domínio específico, processada por um compilador com diagnósticos localizados e acoplada a uma simulação visual, pode oferecer ao estudante **retroalimentação imediata** na construção e na execução de autômatos finitos?



Autosim: o estudante escreve o texto; o diagrama é gerado e animado automaticamente

Objetivos

Objetivo geral: desenvolver uma ferramenta educacional que integra a descrição textual de autômatos, sua compilação pedagógica e sua simulação visual em um único ambiente.

Objetivos específicos

1. Projetar uma **DSL** textual para AFDs e AFNs, com gramática em EBNF;
2. Implementar um **compilador pedagógico** com diagnósticos localizados;
3. Desenvolver um **simulador** de cadeias sobre os autômatos descritos;
4. Construir uma **interface gráfica** que desenhe e anime o autômato;
5. **Validar** a ferramenta com testes, autômatos clássicos e cenários de erro.

Validação **funcional**; a eficácia pedagógica com estudantes é trabalho futuro.

Roteiro

1. Introdução
2. Fundamentação Teórica
3. Trabalhos Relacionados
4. Metodologia de Construção da Solução
5. Ferramentas para Implementação
6. Projeto da DSL
7. Implementação
8. Resultados e Discussões
9. Conclusão

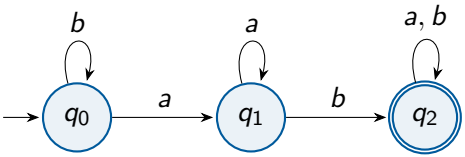
Autômatos finitos

Um **AFD** é uma quíntupla $M = (Q, \Sigma, \delta, q_0, F)$:

- Q: estados Σ: alfabeto
- $\delta : Q \times \Sigma \to Q$: transições
- q_0: inicial $F \subseteq Q$: finais

No **AFN**, $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \to \mathcal{P}(Q)$: vários destinos e transições vazias ε.

AFD e AFN são **equivalentes** (construção de subconjuntos) e reconhecem as **linguagens regulares**.



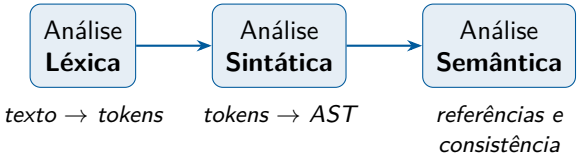
Cadeias que **contêm "*ab*"**

aceita: ab, aab, bab

rejeita: ε, ba, bbb

Reaparece na DSL e na demonstração.

Compiladores e linguagens de domínio específico



DSL (Fowler, 2010)

Linguagem de **expressividade limitada**, voltada a um **domínio específico**.

- **Interna:** embutida em uma linguagem hospedeira (ex.: FSM, em Racket).
- **Externa:** sintaxe própria e **compilador dedicado** — é o caso da **Autosim**.

Roteiro

1. Introdução
2. Fundamentação Teórica
3. Trabalhos Relacionados
4. Metodologia de Construção da Solução
5. Ferramentas para Implementação
6. Projeto da DSL
7. Implementação
8. Resultados e Discussões
9. Conclusão

Panorama: quatro ferramentas gráficas e uma textual

- **JFLAP** — referência mundial; cobre toda a hierarquia de Chomsky; construção **só gráfica**, sem descrição textual nem diagnóstico.
- **GAM** (UFLA) — *desktop*; limitado a 10 estados e 3 símbolos.
- **Simulador UFSC** — *web*, sem instalação; construção gráfica.
- **AutomatoGraph** — só AFDs; gera a gramática regular; simulação parcialmente visual.
- **FSM** (Morazán e Antunez, 2014) — **textual**: autômatos como código Racket, testáveis e versionáveis; muitos formalismos; execução **só em texto**, visualização como trabalho futuro.

Análise comparativa

Trabalho	Descrição textual	Versiona-mento	Compilador/ diagnóstico	Simulação visual	Voltada ao ensino
JFLAP	Não	Não	Não	Sim	Sim
GAM	Não	Não	Não	Sim	Sim
Simulador UFSC	Não	Não	Não	Sim	Sim
AutomatoGraph	Não	Não	Não	Parcial	Sim
FSM	Sim (interna)	Sim	Via Racket	Não	Sim
Autosim	**Sim (externa)**	**Sim**	**Sim**	**Sim**	**Sim**

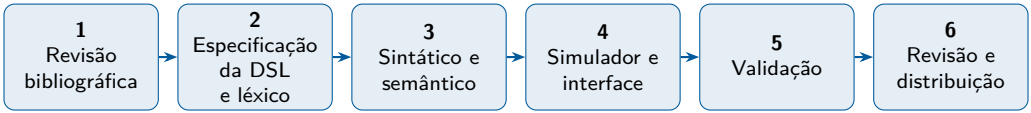
As gráficas **visualizam sem texto**; a textual **descreve sem visualizar**.
A Autosim une as duas coisas, com diagnóstico de compilador.

Roteiro

1. Introdução
2. Fundamentação Teórica
3. Trabalhos Relacionados
4. Metodologia de Construção da Solução
5. Ferramentas para Implementação
6. Projeto da DSL
7. Implementação
8. Resultados e Discussões
9. Conclusão

Metodologia

Pesquisa aplicada, exploratória: pesquisa bibliográfica + desenvolvimento e verificação funcional de um artefato (Gil, 2002; Wazlawick, 2014).



Organização

Dupla com divisão código/texto; reuniões semanais com o orientador; devolutivas incorporadas ao texto e à ferramenta (versões 1.0.1 a 1.0.4).

Critérios de validação

Testes por fase (todos aprovados) · autômatos de referência (veredito = definição formal) · erros induzidos (detectados e localizados).

Roteiro

1. Introdução
2. Fundamentação Teórica
3. Trabalhos Relacionados
4. Metodologia de Construção da Solução
5. Ferramentas para Implementação
6. Projeto da DSL
7. Implementação
8. Resultados e Discussões
9. Conclusão

Por que Rust — e as bibliotecas adotadas

Requisitos atendidos

- **Binário nativo único**: sem instalar JVM ou interpretador.
- **Enumerações com `match` exaustivo**: *tokens*, AST e erros tratados por completo, verificado pelo compilador.
- **Ecossistema** para compiladores com bons diagnósticos.
- **Interface multiplataforma**: Linux, Windows e macOS.

Comparada a Python, Java e C++ (Seção 5.4). Custo: curva de aprendizado.

Biblioteca	**Uso**
Logos 0.16	analisador léxico
Chumsky 0.13	analisador sintático
Ariadne 0.5	diagnósticos de erro
Iced 0.14	interface gráfica
GitHub Actions	testes + pacotes para 3 sistemas

Roteiro

1. Introdução
2. Fundamentação Teórica
3. Trabalhos Relacionados
4. Metodologia de Construção da Solução
5. Ferramentas para Implementação
6. Projeto da DSL
7. Implementação
8. Resultados e Discussões
9. Conclusão

A linguagem: o mesmo autômato, em texto

```
alfabeto { 'a', 'b' }

automato AFD exemplo_afd {
    estados { q0, q1, q2 }
    inicial q0
    finais { q2 }
    transicoes {
        q0 → q1 com 'a'    q0 → q0 com 'b'
        q1 → q1 com 'a'    q1 → q2 com 'b'
        q2 → q2 com 'a'    q2 → q2 com 'b'
    }
}

simular exemplo_afd com "ab"    // aceita
simular exemplo_afd com "bbb"   // rejeita
```

- DSL **externa**, declarativa, em **português**.
- Cada elemento da quíntupla tem uma declaração própria.
- Arquivo `.asl`: alfabeto comum, autômatos e comandos `simular`.
- Texto puro: **versionável** com Git.

Gramática e decisões de projeto

Trecho da gramática em EBNF (completa na Seção 6.2):

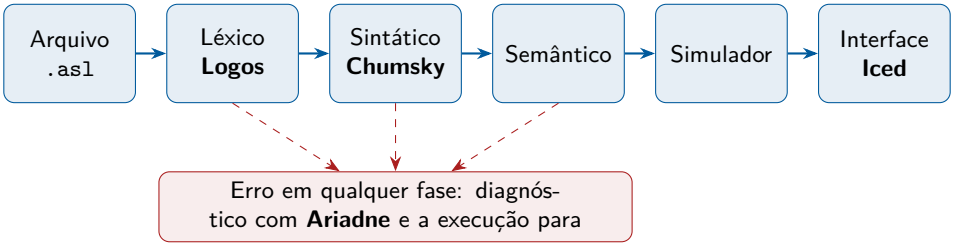
```
declaracao_automato ::= "automato" ("AFD" | "AFN") ident "{"
                        "estados" lista "inicial" ident "finais" lista
                        "transicoes" "{" transicao* "}" "}"
transicao           ::= ident "->" ident "com" (char_literal | "epsilon" | "eps")
comando_simulacao   ::= "simular" ident "com" cadeia_literal
```

- **Alfabeto único** por arquivo.
- Tipo **AFD/AFN explícito**: determinismo só é exigido do AFD.
- **Português** e **case-insensitive** (11 palavras reservadas).
- Transições ε com `epsilon` ou `eps`.

Roteiro

1. Introdução
2. Fundamentação Teórica
3. Trabalhos Relacionados
4. Metodologia de Construção da Solução
5. Ferramentas para Implementação
6. Projeto da DSL
7. Implementação
8. Resultados e Discussões
9. Conclusão

Pipeline de compilação



- **Logos** gera o analisador léxico, um AFD, a partir de anotações: um autômato lendo a linguagem de autômatos.
- **Chumsky** (*parser combinators*): a gramática é escrita compondo funções e produz a AST.

Análise semântica e diagnósticos

1. **Tabela de símbolos**: duplicatas; alfabeto vazio.
2. **Referências**: estados, símbolos e autômatos não declarados.
3. **Determinismo** (só AFD): sem ε e uma transição por (estado, símbolo).

Erros **acumulados**: todos aparecem em uma só execução.

```
Error: token inválido `@`
   [ examples/error_line3.asl:3:22 ]

 3 | automato AFD exemplo @
                          ^ token desconhecido
```

Diagnóstico real da Autosim: arquivo, **linha e coluna**, trecho realçado e descrição em português.

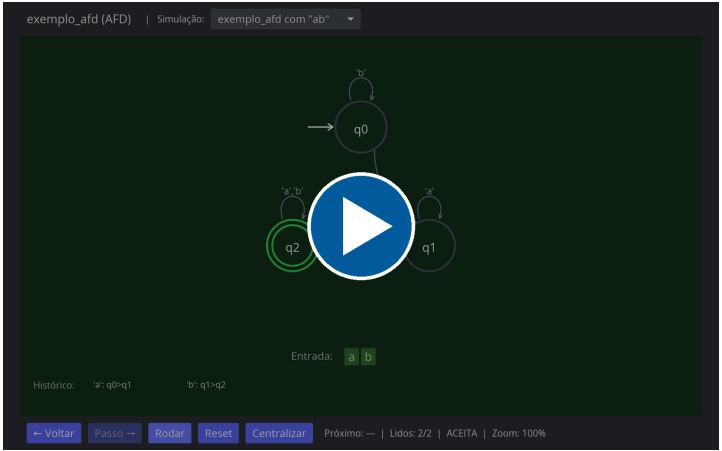
Simulador: um caminho, não uma nuvem de estados

- No AFN, acender **todos os estados ativos** ao mesmo tempo dificultava acompanhar a execução.
- A Autosim exibe um **caminho único**:
  - cadeia aceita $\rightarrow$ um **caminho aceitador**;
  - cadeia rejeitada $\rightarrow$ o caminho que **mais se aproximou** da aceitação.
- Escolhido por **busca em largura** sobre (estado, símbolos consumidos): o veredito continua sendo o da definição formal.
- **Histórico completo**: avançar e voltar passo a passo.

Roteiro

1. Introdução
2. Fundamentação Teórica
3. Trabalhos Relacionados
4. Metodologia de Construção da Solução
5. Ferramentas para Implementação
6. Projeto da DSL
7. Implementação
8. Resultados e Discussões
9. Conclusão

Demonstração



Clique na imagem para reproduzir (1min44s): aceitação e rejeição, diagnósticos de erro, AFN-ε e *pan*/*zoom*.
[abrir no reprodutor de vídeo]

Testes automatizados e cenários de erro

51 testes automatizados

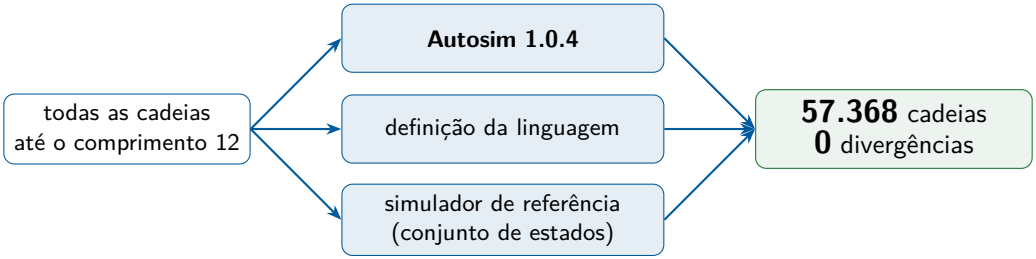
Analisador léxico	12
Analisador sintático	14
Analisador semântico	16
Simulador	9
Total	**51**

Todos aprovados; executados também antes de cada versão publicada.

6 arquivos com erros propositais

- Cada erro foi detectado **na fase correta**: léxica, sintática ou semântica.
- Toda mensagem traz **linha, coluna, trecho realçado** e descrição.
- Um arquivo com **5 erros semânticos** gerou as 5 mensagens em **uma única execução**.

Validação diferencial do simulador



- 8 autômatos clássicos (Hopcroft; Sipser): AFD, AFN e AFN-ε.
- Um AFN e o AFD obtido dele por subconjuntos aceitaram **as mesmas 4.092 cadeias**.
- Limite: exaustiva só até o comprimento testado; verifica o veredito, não o desenho do caminho.

Avaliação preliminar com estudantes

Percepção preliminar — fora do escopo da monografia

Levantamento feito após a entrega do texto, com o guia rápido da ferramenta. Não mede aprendizagem: indica como os estudantes perceberam a Autosim.

- **[PREENCHER]** participantes; período de aplicação.
- **[PREENCHER]** principais resultados (2 ou 3 perguntas).
- **[PREENCHER]** comentário que mais chamou atenção.

Discussão: os objetivos foram atendidos?

Objetivo	**Evidência**
1. DSL em EBNF	Todos os autômatos da validação escritos nela
2. Compilador	42 testes das 3 fases; 6 cenários de erro localizados
3. Simulador	9 testes + 57.368 simulações sem divergência
4. Interface	Demonstrada; **não** submetida a testes automatizados
5. Validação	Os 3 instrumentos aplicados; critérios atendidos

Limites

Casos de teste escritos pelos próprios autores · interface avaliada só pelo uso · a retroalimentação é **produzida corretamente**, mas o efeito sobre a **aprendizagem** não foi medido.

Roteiro

1. Introdução
2. Fundamentação Teórica
3. Trabalhos Relacionados
4. Metodologia de Construção da Solução
5. Ferramentas para Implementação
6. Projeto da DSL
7. Implementação
8. Resultados e Discussões
9. Conclusão

Contribuições

A pergunta de pesquisa foi respondida no **plano funcional**: a Autosim entrega retroalimentação imediata na construção e na execução de autômatos.

- **DSL** declarativa em português para AFDs e AFNs com ε, em EBNF.
- **Compilador pedagógico** com diagnósticos localizados.
- **Simulador** de caminho único, com histórico nos dois sentidos.
- **Interface gráfica** que desenha e anima o autômato a partir do texto.
- Validação: **51 testes**, **57.368 simulações** sem divergência e cenários de erro.
- Distribuída para Linux, Windows e macOS (versão 1.0.4), com manual de uso.

Limitações e trabalhos futuros

Limitações

- Alfabeto único por arquivo.
- Sem modo de linha de comando.
- Um arquivo por execução.
- Sem editor embutido com realce.
- Sem conversão AFN→AFD nem minimização.

Trabalhos futuros

- Editor com realce na interface.
- Conversões AFN→AFD, minimização e expressões regulares.
- Autômatos com pilha e **máquinas de Turing**.
- **Estudos controlados com estudantes**.

Obrigado!

Perguntas?

João Thomaz Vieira · Klaus Siegfried Beckmann

Orientador: Prof. Diógenes Cogo Furlan · UTP, 2026